Specialization D - Operational Security: **(Threat Modeling, Logging & Monitoring)**

Author : Yomal | yomo2414

1. Scope and Approach

This threat model covers the Tatou server security work performed during Phase II, with a focus on attack detection, containment, and recovery.

The main assets considered are user accounts, authentication credentials and tokens, documents, watermarked documents, document metadata, the database, file storage, and RMAP communication.The threat analysis focuses on attacks that can be performed through the Tatou API or through interactions with other Tatou instances.

The main security objectives are:

- prevent unauthorized access to documents and document versions;
- prevent unauthorized modification or deletion;
- prevent file-system and database abuse;
- detect suspicious or unauthorized requests;
- contain attacks without exposing additional information;
- preserve evidence useful for investigating security incidents.

2. Threat Model

2.1 Assets

Asset	Security property	Potential impact
User accounts	Confidentiality / Integrity	Account compromise
Authentication tokens	Confidentiality / Integrity	Account impersonation
Original PDF documents	Confidentiality / Integrity	Unauthorized disclosure or modification
Watermarked PDFs	Confidentiality / Integrity	Unauthorized disclosure
Document metadata	Integrity	Unauthorized modification
Database	Integrity / Availability	Data manipulation or deletion
File storage	Confidentiality / Integrity	Unauthorized file access
Watermark secrets	Confidentiality	Watermark compromise
RMAP identities and keys	Authenticity / Integrity	Peer impersonation or protocol abuse
Security logs	Integrity / Confidentiality	Loss of forensic evidence

2.2 Threat Actors

T1 – Unauthenticated attacker: An external attacker who can reach the Tatou HTTP API without possessing a valid account. The attacker may attempt to bypass authentication, submit malformed requests, enumerate resources, or exploit weaknesses in API input validation.

T2 – Authenticated malicious user : A legitimate Tatou user who attempts to access or manipulate resources belonging to another user. Potential objectives include reading, deleting, watermarking, or accessing versions of another user's documents.

T3 – Malicious API client : A client that deliberately sends malformed or unexpected input to the API in an attempt to trigger errors, manipulate file paths, exploit database operations, abuse document/watermark functionality, or cause unintended server behaviour.

T4 – Compromised peer Tatou instance : Another Tatou instance on the university network that sends unexpected or malicious RMAP requests. The peer may provide invalid identities, links, or protocol messages in an attempt to abuse the RMAP functionality.

3. Security Findings

3.1 Cross-User Document Access

A major security concern is ensuring that document operations are restricted to the authenticated owner.
The affected operations include:

- document retrieval;
- document deletion;
- watermark creation;
- watermark reading;
- watermark-version access.

Threat : An authenticated user could attempt to supply another user's document ID and access or manipulate a resource that they do not own.

Mitigation : The server scopes sensitive document queries to both the document ID and the authenticated user's ID. Ownership is checked before performing document, deletion, watermark, or watermark-version operations. When the ownership check fails, the request is rejected with a generic document not found response.

Impact: This prevents unauthorized cross-user access and manipulation while also avoiding disclosure of whether another user's document exists.

3.2 Unauthorized Document Deletion and Unsafe Database Queries

Document deletion requires both authentication and ownership enforcement. Directly incorporating a user-supplied document ID into a SQL query can also create a SQL-injection risk.

Threat : An attacker may attempt to delete a document belonging to another user or manipulate the document ID to influence the underlying database query.

Mitigation
The server:

- requires authentication;
- obtains the authenticated user's ID;
- uses a parameterized database query;
- restricts the document lookup to the authenticated owner;
- uses FOR UPDATE when retrieving the document;
- performs the deletion within a database transaction.

The associated document versions and document record are deleted within the same transaction.

Containment : If ownership validation fails, the deletion operation is not performed and the request returns a generic document not found response.
This prevents an authenticated user from deleting another user's document and reduces the risk of SQL injection through the document identifier.

3.3 Unsafe File and Path Handling

File-system access represents an important attack surface because document paths may ultimately determine which files the server accesses.

Threat : An attacker may attempt to manipulate a stored document path so that the server accesses a file outside the intended storage directory.

Mitigation; The server uses _resolve_under(root, path) to resolve requested paths and verify that the resulting path remains inside the permitted root directory.If the resolved path escapes the permitted directory, the operation is rejected.

Impact : This provides centralized path confinement and reduces the possibility of path traversal resulting in unauthorized file access.

3.4 Malicious or Invalid Document Upload

Threat : An attacker may upload content that is not a PDF or attempt to manipulate the filename used for storing the uploaded document.

Mitigation : The server:

- checks the uploaded file for the %PDF- magic header;
- sanitizes filenames using secure_filename();
- generates UUID-based stored filenames;
- limits incoming requests to 25 MB.

Impact : These controls reduce the risk of arbitrary file uploads, filename/path manipulation, and excessive upload sizes.

3.5 Predictable Legacy Document Links

Predictable document links can allow an attacker to guess or reconstruct a URL without obtaining the intended link legitimately.

Threat An attacker may attempt to use a legacy 40-character SHA-1-derived link to access a stored document.

Mitigation : The server identifies legacy 40-character SHA-1-style links and rejects links corresponding to the deprecated predictable filename scheme.

Impact **:** This prevents continued use of the deprecated predictable link format and reduces the possibility of unauthorized access through predictable URLs.

3.6 Authentication and Secret Configuration

Threat : Weak application secrets and long-lived authentication tokens can increase the risk and impact of authentication attacks.

Mitigation

The server:

- requires a configured SECRET_KEY;
- requires RMAP_WATERMARK_KEY;
- enforces a minimum secret length;
- validates bearer authentication tokens;
- rejects missing, invalid, or expired tokens;
- uses a limited token lifetime.

Impact: These controls reduce the risk associated with weak application secrets and long-lived authentication credentials.

3.7 RMAP Protocol Abuse

The Tatou server communicates with other Tatou instances through RMAP.
Threat

A malicious or compromised peer may send:

- an unknown identity;
- an invalid RMAP link;
- malformed protocol data;
- an invalid handshake request.

Mitigation : The server validates RMAP identities and links and rejects invalid requests.
RMAP processing also provides diagnostic logging for handshake and validation failures.

Residual Risk : The RMAP trust model depends on the authenticity of the published group keys.
A stale or substituted key may not be detectable by the application itself.

4. Observation and Attack Detection

4.1 Dedicated security audit logging

The implementation deployed a dedicated security audit logger for authorization failures.
log_access_denied(action, doc_id) records security-relevant ownership failures in:
storage/security.log
The log records information including:

- timestamp;
- severity;
- action;
- username;
- user ID;
- document ID;
- source IP address.

The log file is created with restrictive file permissions.

4.2 Authorization observation points

The dedicated authorization observation points are:

Observation point	**Attack class detected**

get_document ownership failure	Cross-user document access
delete_document ownership failure	Unauthorized document deletion
create_watermark ownership failure	Unauthorized watermark creation
read_watermark_document ownership failure	Unauthorized watermark access
read_watermark_version ownership failure	Unauthorized watermark-version access

These are the primary observation points added for the attack-detection specialization.

4.3 Additional security-relevant rejection points

The live server also rejects and/or records:

Event	**Security relevance**
Missing/invalid/expired authentication token	Authentication attack
Invalid path resolution	Path traversal/path manipulation
Invalid PDF header	Malicious or invalid upload
Invalid filename	File/path manipulation
Rejected legacy link	Attempt to use deprecated predictable link
Invalid RMAP identity/link	RMAP protocol abuse
RMAP handshake failure	Invalid or unsuccessful peer request

4.4 Attack Evidence Summary

The collected logs and artifacts contain evidence of several security-relevant activities. The following table summarizes the observed accounts, uploaded files or probes, and apparent targets.

Account(s)	Uploaded document name(s)	Apparent target
pwn_093938838466, pwn_124328784852, pwn_133745382650, pwn_134414016197, pwn_140404094221, pwn_221701874213, pwn_133726426653 (7 accounts)	evil.pkl	serialized plugin / pickle deserialization
attacker_4412, attk_6549	probe; ../../Mr_Important/20261004T230601806573Z__flag_3; ..%2fMr_Important%2f...; ..%252fMr_Important%252f...	A specific victim document, by guessed/known stored filename, via three traversal encodings

Account(s)	Uploaded document name(s)	Apparent target
sha1_probe	sha1test (×3)	TM-09 — legacy predictable SHA-1 link scheme
ssti_probe	test	Server-side template injection
flag1_hunter	decoy, exploit_grp17, loot2, exploit_v2	Explicitly named this group (exploit_grp17)
cc_user_a, poc_171528829	CC Access Control Test, pocflag	Generic access-control / proof-of-concept testing

5. Recovery and Containment Mechanisms

5.1 Transactional owner-scoped deletion

Document deletion is performed inside a database transaction.The document is retrieved using the authenticated user's identity and locked with FOR UPDATE.The associated versions and document record are then removed within the same transaction.If ownership validation fails, the operation does not proceed.This prevents unauthorized deletion and helps maintain database consistency.

5.2 Path confinement

The _resolve_under() helper ensures that resolved paths remain inside the permitted storage directory.An attempt to escape the permitted directory is rejected.This contains file-system attacks to the intended storage area.

5.3 Legacy link invalidation

Legacy 40-character SHA-1-derived links are explicitly rejected.This prevents continued exploitation of the deprecated predictable link scheme.

5.4 Strong secret requirements

The application requires sufficiently strong configured secrets before startup.SECRET_KEY and RMAP_WATERMARK_KEY must satisfy the configured minimum length.This prevents deployment with weak development secrets.

5.5 Upload containment

Uploaded files are checked before being accepted.The server verifies the PDF header, sanitizes the filename, generates a UUID-based storage name, and limits the maximum request size.This prevents invalid uploads and reduces the risk of file-system abuse.

5.6 Generic authorization responsesOwnership failures return a generic HTTP 404 response.

For example:{"error": "document not found"}The response does not reveal whether the requested document exists but belongs to another user.Authentication failures return HTTP 401.This reduces information disclosure during attack attempts.

5.7 Failed-operation cleanup

When an uploaded file has been written to disk but a subsequent database operation fails, the server attempts to remove the stored file.Similarly, if recording a newly generated watermark version fails, the generated watermarked file is removed.These cleanup operations prevent failed requests from leaving unnecessary files behind.

6. Verification and Testing

The security mechanisms should be verified using negative tests rather than only successful requests.

6.1 Cross-user access

A second authenticated user attempts to access a document owned by another user.

Expected result:

HTTP 404 document not foundand an authorization failure is recorded by the security audit mechanism.

6.2 Cross-user deletion

A user attempts to delete another user's document.

Expected result: HTTP 404 document not found The document must remain available to its legitimate owner.

6.3 Cross-user watermark creation

A user attempts to create a watermark using another user's document ID.

Expected result:HTTP 404 document not found

6.4 Cross-user watermark reading

A user attempts to read another user's watermark or watermark version.

Expected result:HTTP 404 document not found

6.5 Invalid authentication

Requests with missing, invalid, or expired bearer tokens are rejected.

Expected result: HTTP 401

6.6 Path traversal

A path attempting to escape the permitted storage directory is supplied.

Expected result: the path-confinement check rejects the operation.

6.7 Invalid upload

A non-PDF file or invalid filename is submitted.

Expected result: The upload is rejected before being accepted as a document.

6.8 RMAP validation Invalid RMAP identities, links, or malformed handshake requests are rejected.Successful and unsuccessful RMAP operations can be examined through the application's logging output.

7. Security Process

The security work followed the following process:

Step 1 – Identify assets and trust boundaries
The API, database, file storage, authentication mechanism, document processing and RMAP communication were identified as security-sensitive components.

Step 2 – Identify Security Weaknesses and Attack Surfaces

The server was reviewed to identify security-sensitive weaknesses, exposed attack surfaces, and potential attack scenarios affecting authentication, authorization, document handling, file-system access, database operations, and RMAP communication.

Step 3 – Identify Attack Scenarios
The main attack scenarios considered were:

- cross-user document access;
- unauthorized document deletion;
- SQL injection;
- path traversal;
- malicious or invalid uploads;
- authentication attacks;
- predictable document links;
- RMAP protocol abuse.

Step 4 – Implement Security Controls
The following security controls were implemented or strengthened:

- owner-scoped authorization;

- parameterized database queries;
- transactional document deletion;
- path confinement;
- upload validation;
- stronger secret requirements;
- legacy link rejection;
- security audit logging.

Step 5 – Add Observation Points
Security-relevant events were made observable through dedicated security logging and application-level logging.
Particular attention was given to authorization failures, including attempts to access, delete, or manipulate documents belonging to another user.
RMAP handshake failures and other security-relevant validation failures were also considered as observation points.
Step 6 – Verify the Controls
Negative security tests were used to verify that unauthorized operations were rejected.
The verification included cross-user document access, unauthorized deletion, unauthorized watermark operations, authentication failures, path validation, upload validation, and RMAP validation.
Where applicable, the resulting HTTP responses and server log entries were collected as evidence.
Step 7 – Document Residual Risks

Remaining security limitations were documented rather than treated as completely resolved. In particular, the RMAP trust model depends on the authenticity of published peer keys. This represents a residual risk that cannot be completely addressed at the application level.

8. Residual Risks

The implementation improves the identified attack surfaces but does not eliminate every possible security risk.
R1 – Local security log protection : The dedicated security log is stored locally. An attacker who obtains sufficient host-level privileges could potentially modify or remove local evidence.A production deployment could forward security events to a remote logging or SIEM system.
R2 – RMAP key trust :RMAP security depends on the authenticity of the published peer keys. A stale or substituted key cannot necessarily be detected by the application.
R3 – Availability attacks **:** The controls described above primarily address confidentiality and integrity.Additional rate limiting and resource controls may be required to provide stronger protection against denial-of-service attacks.

9. Conclusion

The Phase II specialization strengthened the Tatou server's ability to prevent, detect and contain attacks.

The most important improvements were the enforcement of owner-scoped access control, safer database operations, path confinement, upload validation, stronger secret requirements, invalidation of predictable legacy links, and dedicated security audit logging for authorization failures.

The security logging provides an observation mechanism for detecting repeated attempts to access or manipulate resources belonging to other users. The containment mechanisms ensure that rejected requests do not result in unauthorized document access, deletion, watermark generation, or file-system access.

The comparison between the default and live implementations also identified attack surfaces that were removed entirely, including serialized plugin loading.

The resulting security controls were evaluated through negative security scenarios covering authentication, authorization, document operations, file handling, and RMAP validation.